MoveInSync Backend Case Study 2026

SDE – Java Backend Hiring Assessment · Case Study 6

Case Study 6: Driver Duty Roster & Assignment

Every shift, drivers must be paired with vehicles and trips — but not any driver with any vehicle. A bus needs a heavy-licence driver, a driver must rest at least 8 hours between duties, and nobody with an expired medical or licence may drive. Build the engine that produces a valid duty roster from a pool of drivers and vehicles and flags every conflict it cannot satisfy.

I. Drivers, Vehicles & Duties

- Driver — licence class (LMV / HMV), medical & training validity with expiry dates, phone.
- Vehicle — type (CAB / BUS / SHUTTLE), each requiring a matching licence class.
- Duty — driver + vehicle + shift window (start/end); status ACTIVE / COMPLETED / CANCELLED.

II. Valid Assignment

- Licence–Vehicle Match — a BUS duty needs an HMV driver; reject or re-assign otherwise.
- Rest-Hour Rule — at least 8 hours between a driver's duty-end and the next duty-start; never roster a breach.
- No Overlap — a driver cannot hold two overlapping duties, and a vehicle cannot be in two duties at once.
- Eligibility — block assignment if the driver's medical / licence / training is expired on the duty date.

III. Build the Roster

- Assign the Pool — given the required duties for a day/week and a pool of drivers + vehicles, produce a feasible assignment that minimises idle drivers and overtime.
- Reassign on Dropout — if a driver calls in sick, re-assign from the pool without breaking any rule, keeping an audit trail.

IV. Reporting

- Conflicts & Utilisation — list any duty that cannot be filled and show each driver's / vehicle's utilisation over the period.

Example Use Case

• A 06:00–10:00 BUS duty: driver D7 (HMV, medical valid) is free and rested 10 h → assigned; D3 (LMV) is skipped as ineligible for a bus.

• D7 then has a 12:00 duty but only 2 h rest after a long morning → the roster refuses it and finds another driver.

Key Points to Keep in Mind

- Model duties as intervals and get the overlap and rest-gap tests exact.
- State your assignment strategy (greedy, bipartite matching, etc.) and its complexity; a correct greedy with clear rules beats a vague "optimiser".
- Expiry and licence-class checks must gate assignment before anything else.
- Show a clean, explainable result when a duty genuinely cannot be filled — don't silently drop it.

Plus Points in Implementation (Overall Evaluation Criteria)

1. **Authentication:** Implement robust user authentication and role-based access (employee vs admin vs approver) to ensure secure access.
2. **Cost Estimation – Time and Space:** Analyse the time and space complexity of your core logic. Use efficient algorithms and data structures to optimise both.
3. **Handling System Failure Cases:** Implement fault-tolerant mechanisms, backup/recovery and clear error-recovery procedures to minimise downtime and protect data integrity.

4. **Object-Oriented Programming (OOPS):** Choose a robust OOPS language (Java preferred). Apply encapsulation, inheritance and polymorphism for maintainable, extensible code with clean controller/service/repository layering.
5. **Trade-offs in the System:** Clearly define and document the trade-offs you made. Explain the rationale behind your architectural and design decisions (performance vs scalability vs maintainability).
6. **System Monitoring:** Add monitoring/metrics and structured logging so issues can be spotted quickly; expose key counters (e.g. rejections, violations, promotions).
7. **Caching:** Integrate caching for frequently-accessed/hot data to reduce database load, with a sensible eviction policy — and say what becomes stale.
8. **Error and Exception Handling:** Build a robust error/exception-handling framework with meaningful, non-leaking error messages for effective debugging.

Instructions

1. **Read and Understand the Problem Statement:** Carefully read the case study. Understand the requirements, inputs, expected outputs and the rules.
2. **Choose a Programming Language:** Select a language you are comfortable with. An object-oriented language (Java preferred; C#, Python acceptable) is expected for a backend role.
3. **Design Your Solution:** Plan first. Decide your data model (ER diagram), the main classes, and the APIs before writing code.
4. **Write the Code:** Write clean, layered code (controller / service / repository) with meaningful names. Break logic into small, testable functions.
5. **Test Your Code:** Test normal inputs AND edge cases (empty data, bad input, duplicate/concurrent requests). Add unit tests for the important logic.
6. **Document Your Code:** Add a README: how to run it, your assumptions, and a short note on the choices/trade-offs you made.
7. **Submit Your Solution:** Push to a public GitHub repository and share the link. Keep a meaningful commit history — not one giant commit.
8. **Demonstration:** Add a short demo video or screenshots showing the main features and a few sample API calls (Postman / curl).